

Execução e Análise de Falha

Ao executar a função `fatorial_quebrado(1)`, o programa falha com a seguinte mensagem:

```
AssertionError: Erro: invariante violado no corpo do loop
```

A falha ocorre na asserção de manutenção do invariante, dentro do corpo do comando `while`, na linha:

```
assert f == math.factorial(i) and 0 <= i <= n, "Erro: invariante violado no corpo do loop"
```

Essa asserção verifica se, após uma iteração do laço, o invariante permanece verdadeiro. O invariante considerado é que `f` deve ser igual a $i!$, além de satisfazer a condição $0 \leq i \leq n$.

Para a entrada $n = 1$, a execução se inicia com $f = 1$ e $i = 0$. Nesse estado inicial, o invariante $f = i!$ é verdadeiro, pois $1 = 0!$. Além disso, a relação $0 \leq i \leq n$ também é satisfeita, uma vez que $0 \leq 0 \leq 1$.

Como a condição do laço $i < n$ também é verdadeira, isto é, $0 < 1$, o programa entra no corpo do `while`. Nesse ponto, o código executa a linha com o bug original:

```
f = f * i
```

Como i vale 0 nesse momento, o cálculo realizado é:

$$f = 1 \cdot 0 = 0.$$

Em seguida, o programa atualiza o valor de i :

```
i = i + 1
```

Assim, i passa a valer 1. Após essa atualização, a asserção de manutenção verifica se `f == math.factorial(i)`, ou seja, se:

$$0 = 1!.$$

Como $1! = 1$, a comparação efetiva passa a ser:

$$0 = 1.$$

Essa igualdade é falsa. Portanto, o invariante de loop não é preservado após a primeira iteração, e o programa levanta o `AssertionError` indicado.

A razão aritmética da falha está no fato de que o código original multiplica f por i antes de incrementar i . Como o laço começa com $i = 0$, o valor acumulado do fatorial é zerado logo na primeira iteração. Consequentemente, após a atualização das variáveis, f deveria representar $i!$, mas passa a valer 0 enquanto i vale 1. Isso caracteriza a violação do invariante de manutenção no corpo do laço.